A

Acadex
The technical learning guide

How an AI-native, multi-tenant school and university platform is built — architecture, security, data, features, AI, frontend, testing and deployment — with the files to read for every part.

Har chapter ke shuru mein Roman Urdu mein khulasa.

Generated from `docs/learning-guide.md` · 2026-10-19

How Acadex is built, part by part, and why each decision was made. Read it front to back once, then use it as a map: every section names the files to open next to it. File paths are relative to the repository root.

> **Is guide ko kaise parhein (Roman Urdu):** Har chapter ke shuru mein ek chhota sa khulasa hai. Pehle khulasa parhein, phir tafseel, phir bataye gaye files code editor mein khol kar saath saath dekhein. Seekhne ka sab se achha tareeqa: ek chhota sa feature khud banayein — Chapter 18 mein qadam ba qadam tareeqa diya gaya hai.

Contents

1. The big picture
2. Running it on your computer
3. The life of an API request
4. Multi-tenancy: keeping every institution's data apart
5. The database: schema, migrations, seed
6. Authentication: passwords, sessions, tokens, 2FA, SSO
7. Authorization: roles and permissions
8. The business features, one by one
9. Background jobs, schedules and live updates
10. Files and storage
11. The AI layer
12. The web app (frontend)
13. The shared package
14. Testing
15. Security, layer by layer
16. Build, CI/CD and deployment
17. Real bugs found while building — and what they teach
18. Recipe: adding a feature end to end
19. Glossary
20. Where to read next

1. The big picture

> **Khulasa:** Acadex ek "multi-tenant SaaS" hai — yani ek hi software bohat se schools, colleges aur universities istemal karte hain, aur har ek ka data doosre se bilkul alag rehta hai. Teen hisse hain: **API** (backend, Node.js + Express + PostgreSQL), **web app** (React), aur **shared** package (dono ke beech saanjha schemas). Teeno ek hi repository mein hain (monorepo).

What it does

Acadex runs an educational institution end to end: structure (campuses, classes, sections), people (students, staff, guardians), teaching (courses, assignments, quizzes, gradebook), attendance and timetables, exams and results, fees and online payments, admissions, messaging and notifications, optional modules (library, HR & payroll, transport, hostel, inventory, visitors), AI assistance (tutor, teacher copilot, grading help, early warning) — and, for the platform owner, the SaaS business around it (plans, billing, usage, API management).

Who uses it

Role	Scope	What they do
Platform administrator (`super_admin`)	the whole platform	creates institutions, plans, billing, AI models, support sign-in
Group administrator (`group_admin`)	a group of institutions	reads every campus of a chain
Institution administrator (`institution_admin`)	one institution	runs everything inside it
Teacher, student, parent	one institution	their own classes, work, children
Custom roles	an institution or a campus	any set of permissions (accountant, exam controller…)

The architecture in one picture

```
Browser (React SPA, PWA)
   │  HTTPS, JSON, Bearer access token + httpOnly refresh cookie
   ▼
API (Express 5, TypeScript)  ── pg-boss workers (same process or `npm run worker`)
   │  one transaction per request, Row-Level Security on every tenant table
   ▼
PostgreSQL (Neon in the cloud; embedded Postgres in tests)
   │
   ├─ Files: local disk, or S3 / MinIO / Cloudflare R2
   ├─ Email (SMTP), web push, WhatsApp (optional: Meta, Twilio)
   ├─ Payments (JazzCash, Easypaisa, Stripe, bank transfer)
   └─ AI providers (Gemini, Groq)
```

The repository

```
apps/api/            the backend (Express + Drizzle + pg-boss)
  src/app.ts         builds the Express app: middleware order and every route mount
  src/server.ts      starts it (and the job workers)
  src/config.ts      every environment variable, validated with Zod at start-up
  src/db/            schema (Drizzle), migration runner, request context, seed
  src/auth/          sessions, tokens, MFA, invites, email tokens, crypto helpers
  src/middleware/    auth middleware (who is calling, which permissions)
  src/routes/        one file per area (people, finance, exams, ai, platform-ai …)
  src/lib/           domain logic shared by routes (ledger, notify, webhooks, lti …)
  src/ai/            the AI layer (client, catalog, providers, prompts, tutor …)
  src/jobs/          background job queue
  drizzle/           generated SQL migrations (+ hand-written RLS / audit SQL)
  tests/             1,678 API tests (Vitest + Supertest + embedded Postgres)
apps/web/            the frontend (React 18 + Vite + TanStack Query + Tailwind 4)
  src/App.tsx        all routes
  src/components/    AppShell (layout + menu), ui/ (design system), shared widgets
  src/lib/           api client, auth context, formatting, device (PWA), i18n
  src/pages/         one folder per area (admin, lms, office, platform, site …)
  public/sw.js       the service worker (offline + push)
packages/shared/     Zod schemas, permissions, constants, countries — used by both
e2e/                 Playwright end-to-end tests (desktop and 360 px phone)
cloudflare/          Worker + wrangler config for hosting on Cloudflare
docs/                architecture, security, operations, AI, platform, Cloudflare, checklist
scripts/leak-scan.mjs  credential scanner run before every commit
ops/                 backup / restore scripts
```

The stack, and why

Piece	Choice	Why
Language	TypeScript everywhere	one language, types shared between API and web
API framework	Express 5	small, well known; Express 5 handles async errors natively
Database	PostgreSQL	relational data, transactions, Row-Level Security, LISTEN/NOTIFY, full-text search
ORM	Drizzle (schema + migrations), raw SQL for queries	the schema is typed and migrations are generated, but queries stay plain SQL you can read
Validation	Zod 4	one schema validates the request *and* types it; shared with the web forms
Jobs	pg-boss	a job queue inside the same Postgres — no Redis to run
Frontend	React 18 + Vite	fast dev server and builds; huge ecosystem
Server state	TanStack Query	caching, refetching, loading/error states for every API call
Forms	react-hook-form + the shared Zod schemas	the form uses the exact rules the API enforces
Styling	Tailwind CSS 4 + semantic colour tokens	light/dark themes from one set of tokens
Tests	Vitest, Supertest, embedded-postgres, Playwright, axe-core	real database in unit tests, real browser in E2E, accessibility checked automatically
Build	tsup (API bundle), Vite (web)	small, fast production bundles

2. Running it on your computer

> **Khulasa:** Node 22 aur PostgreSQL chahiye (ya Docker se Postgres). `.env` file banayein, migrations chalayein, demo data daalein, phir `npm run dev` se API (port 5000) aur web (port 3000) dono chal jate hain.

```
npm ci                                  # install every workspace
cp apps/api/.env.example apps/api/.env  # then set DATABASE_URL and JWT_SECRET
docker compose up -d                    # optional: Postgres + MinIO + Mailpit locally
npm run db:migrate                      # apply migrations
npm run db:seed                         # demo institutions and accounts (password Acadex@123)
npm run dev                             # API on :5000, web on :3000 (proxying /api)
```

Useful scripts (root `package.json`): `test`, `lint`, `typecheck`, `build`, `e2e`, `secrets:check`. API-only: `db:generate` (new migration from the schema), `db:studio`, `worker` (job workers in their own process), `ai:eval:real` (AI evals against a real model).

`apps/api/src/config.ts` is the single place every setting is read. It parses `process.env` with a Zod schema and **refuses to start** with a list of problems if something required is missing or wrong (for example `MFA_ENCRYPTION_KEY` in production, or `CRON_SECRET` shorter than 32 characters). Lesson: validate configuration once, at start-up, loudly.

---

3. The life of an API request

> **Khulasa:** Har request ek tay shuda silsile se guzarti hai: logging → security headers → CORS → body parsing → rate limit → database transaction kholna → login check → permission check → route ka kaam → transaction commit → jawab. Ye silsila `apps/api/src/app.ts` mein hai.

Open `apps/api/src/app.ts` and follow along. The order of `app.use(...)` *is* the request's path:

1. `httpLogger` (pino) — one structured log line per request, with a request id.
2. `helmet()` — security headers (`X-Content-Type-Options: nosniff`, frame options, …).
3. `cors(...)` — only the origins in `CORS_ORIGINS` may call with credentials.
4. `Cache-Control: no-store` on every `/api` answer — personal data must never sit in a browser or proxy cache (see Chapter 17 for the bug that led here).
5. **Raw bodies for webhooks** (`/api/payments/webhook`, Stripe, Zoom) — signatures are computed over the exact bytes, so these routes must not be JSON-parsed first.
6. `express.json({ limit: '100kb' })` and `cookieParser()`.
7. **Rate limits** — a general per-IP limit, a strict one on login, MFA, password reset, sign-up; AI calls limited per person.
8. `dbContext` (`src/db/context.ts`) — opens the request's database context (Chapter 4).
9. **Routers** — each router declares `auth` and `requirePermission('…')` on its routes.
10. `errorHandler` (`src/lib/errors.ts`) — turns thrown errors into JSON.

A route, read slowly

Most routes look like this (simplified from `src/routes/bookings.ts`):

```
router.post('/room-bookings', requireAny('teaching.access', 'timetable.manage'), async (req, res) => {
  const b = roomBookingBody.parse(req.body);          // 1. validate (Zod, from packages/shared)
  const inst = institutionId(req);                    // 2. the caller's institution
  const out = await withTransaction(async () => {     // 3. a savepoint inside the request transaction
    const { rows } = await db.query(`SELECT … FOR UPDATE`, [b.room_id, inst]);  // 4. lock, check
    if (!rows[0]) throw badRequest('Choose one of your rooms');
    …
    return (await db.query(`INSERT … RETURNING *`, [...])).rows[0];
  });
  res.status(201).json(out);                          // 5. answer
});
```

What to notice:

- **Validation first.** `roomBookingBody` lives in `packages/shared` so the web form validates with the same rules. A `ZodError` becomes `400` with the field errors.
- **Errors are thrown, not returned.** `badRequest`, `notFound`, `conflict`, `forbidden` create an `HttpError`; Express 5 passes thrown errors from async handlers to the error handler.

- **Plain SQL with parameters** (`$1, $2`) — never string concatenation, so no SQL injection.
- `FOR UPDATE` locks the row so two people can't book the same room at the same moment.
- **No commit call.** The request's transaction commits automatically just before the response is sent, and rolls back for any error status.

Pagination

`src/lib/pagination.ts` : `parsePage(req.query)` reads `page` / `pageSize` (max 100), the SQL adds `count(*) OVER() AS _total` , and `sendPage(res, rows, pg)` returns the rows with `X-Total-Count` , `X-Page` and `X-Page-Size` headers. The web's `usePagedList` hook reads those headers.

API documentation

`src/openapi.ts` plus the generated `src/openapi-lms.ts` describe every route (486 of them); `/api/docs` renders them. `apps/api/scripts/gen-openapi-lms.ts` regenerates the file from the route files, and a test fails if a route file is not mounted or documented.

---

4. Multi-tenancy: keeping every institution's data apart

> **Khulasa:** Saare schools ek hi database mein hain. Data alag rakhne ke liye do deewarein hain: (1) har query mein `institution_id` ka filter, aur (2) PostgreSQL ki **Row-Level Security** — agar developer filter bhool bhi jaye, database khud doosre school ki rows chhupa deta hai. Ye is project ka sab se aham security design hai.

The tenant hierarchy

`platform → group (optional chain) → institution → campus → department → program → level → section`. Almost every table has an `institution_id`. Role assignments carry a scope: platform, group, institution or one campus.

Layer 1: every query filters by institution

Routes read the caller's institution with `institutionId(req)` and pass it as a parameter. Every referenced id (a section, a student) must belong to that institution — checked in the route.

Layer 2: Row-Level Security (RLS)

Migration `drizzle/0002_row_level_security.sql` and later ones turn on RLS for every tenant table:

```
ALTER TABLE students ENABLE ROW LEVEL SECURITY;
ALTER TABLE students FORCE ROW LEVEL SECURITY;          -- applies to the table owner too
CREATE POLICY tenant_isolation ON students
  USING      (app_bypass_rls() OR institution_id = app_institution_id())
  WITH CHECK (app_bypass_rls() OR institution_id = app_institution_id());
```

`app_institution_id()` reads the transaction-local setting `app.institution_id`; `app_bypass_rls()` reads `app.bypass_rls`. The application connects as a normal (non-superuser) role, so the policies always apply.

How the settings get there: `src/db/context.ts`

This file is worth reading slowly. It uses Node's `AsyncLocalStorage` to give each request its own "store" without passing it through every function:

- `dbContext` middleware creates the store and opens **one transaction on one pooled connection** for the request.
- After authentication, `setTenant(institutionId, userId)` records who is calling.
- Before each query, the context applies `set_config('app.institution_id', …, true)` (the `true` makes it transaction-local), so RLS sees the right institution.
- `res.end` is wrapped: the transaction **commits before the response is sent** — a failed commit can never look like success — and rolls back for error responses. If the client disconnects, the transaction is rolled back and the connection released.

- `withTransaction(fn)` makes a savepoint inside the request transaction.
- `afterCommit(fn)` runs code only once the transaction has committed (queue a job, publish a live event) — so nothing is announced for work that was rolled back.
- `asSystem(fn)` / `runAsSystem(fn)` bypass RLS deliberately for platform work, sign-in lookups and jobs; `systemContext` does it for a whole platform router.

Lesson: making the safe path the default (RLS on, filter forgotten = no rows) is better than relying on every developer remembering.

Soft delete, recycle bin and audit

- Important records are soft-deleted (`deleted_at`); `src/lib/soft-delete.ts` and the recycle bin restore them; a nightly job purges old ones.
- An `acadex_audit()` trigger (migration `0008`) writes every insert/update/delete of audited tables to `audit_log` with the old and new values, the user and the request id; sensitive tables use `acadex_audit('masked')` so secrets are never copied into the log. Sensitive *reads* (medical notes, transcripts, exports) are audited by the routes.

---

5. The database: schema, migrations, seed

> **Khulasa:** Tables TypeScript mein `apps/api/src/db/schema/` ke andar likhe hain (Drizzle). `npm run db:generate` un se SQL migration banata hai; RLS aur triggers jaisi cheezein hum haath se usi file ke aakhir mein jodte hain. Migration runner har file ek dafa, checksum aur lock ke saath chalata hai, aur khatarnak (data mitane wali) SQL ko rok deta hai.

- **Schema** — `src/db/schema/*.ts` (185 tables), e.g. `core.ts` (institutions, users, plans), `academics.ts`, `lms.ts`, `finance.ts`, `ai.ts`, `platform.ts`. Each table declares its foreign keys, unique constraints, check constraints and indexes with explicit names.
- **Generating a migration** — change the schema, run `npm run db:generate -w @acadex/api -- --name my_change`; drizzle-kit writes `drizzle/00NN_my_change.sql` and a snapshot. Then append the hand-written parts: RLS policies for new tenant tables, audit triggers, data backfills.
- **The runner** — `src/db/migrate.ts`: each file runs once inside a transaction, is recorded in `schema_migrations` with a checksum (editing an applied file is detected), runs under a **Postgres advisory lock** (two deploys can't migrate at once), and **refuses destructive statements** (`DROP TABLE`, `DROP COLUMN`, `TRUNCATE` …) unless the file carries the marker `-- acadex:reviewed-destructive`. Forward only — there are no down migrations; fixes are new migrations.
- **Schema drift test** — a test compares the database built by the migrations with the Drizzle schema (names of constraints included), so they can't silently diverge.
- **Seed** — `src/db/seed.ts` builds four demo tenants (a school, a three-campus chain, a university, a training centre) with realistic data; it refuses `--reset` in production.

---

6. Authentication: passwords, sessions, tokens, 2FA, SSO

> **Khulasa:** Login par do cheezein milti hain: 15 minute ka **access token** (JWT, har request ke saath) aur ek **refresh token** (httpOnly cookie, JavaScript nahi parh sakta). Access token khatam ho to refresh token naya access token deta hai — aur har dafa refresh token bhi badal jata hai. Agar purana refresh token dobara aaye to samjho chori hua, poora session band.

Files: `src/auth/service.ts`, `sessions.ts`, `tokens.ts`, `mfa.ts`, `totp.ts`, `crypto.ts`, `src/middleware/auth.ts`, `src/routes/auth.ts`, `src/lib/passwords.ts`.

Passwords

Hashed with **bcrypt** (10 rounds); at least 10 characters (shared schema). Plain passwords are never stored or logged. Accounts are created by **invitation** (a single-use emailed link) rather than by an admin typing a password.

Access and refresh tokens

- **Access token**: a JWT signed with `JWT_SECRET`, valid 15 minutes (`ACCESS_TOKEN_TTL`), sent as `Authorization: Bearer …`. It carries the user id, institution, session id and a `token_version`.
- **Refresh token**: a random string, stored only as a SHA-256 hash, sent as the httpOnly cookie `acadex_rt` scoped to `/api/auth`. `POST /api/auth/refresh` returns a new access token **and a new refresh token**, marking the old one used. **Reuse detection**: if a used refresh token is presented again, it was copied, so the whole session is revoked.
- **CSRF**: cookie-authenticated endpoints require the header `X-Acadex-Client: web` — a cross-site form post can't add a custom header.
- **Revocation**: every request re-loads the user and session (`middleware/auth.ts`): an inactive user, a revoked or expired session, a changed `token_version` (password change, "sign out everywhere") or a suspended institution is refused at once, even with a valid JWT.

Two-factor authentication (2FA / MFA)

TOTP (RFC 6238, 30-second steps, 6 digits) compatible with Google/Microsoft Authenticator:

1. Setup creates a random secret, **encrypted with AES-256-GCM** (`MFA_ENCRYPTION_KEY`), and shows a QR code (`otpauth://` URI).
2. Enabling requires one valid code; ten **recovery codes** are shown once and stored hashed.
3. Sign-in with 2FA on returns only a 5-minute challenge token; the code turns it into a session.
4. Each code works once (`last_used_step` stops replays); ±1 step allows clock drift; the verify endpoint is rate limited.
5. Administrators (`super_admin`, `group_admin`, `institution_admin`) **must** enrol when `ADMIN_MFA=required` (the production default); the API refuses everything else until they do.

Single sign-on

Google and Microsoft sign-in (`src/lib/sso.ts` , `src/routes/sso.ts`) use OpenID Connect with **PKCE** and a signed state cookie; only existing accounts can sign in (no self-created users), and 2FA still applies after SSO.

Support sign-in (impersonation)

The platform owner can sign in as an institution's admin only if that institution allows support access; the session lasts one hour, shows a banner, and is written to both audit logs.

---

7. Authorization: roles and permissions

> **Khulasa:** Har kaam ke liye ek "permission" hai (jaise `fees.manage`). Roles permissions ka majmua hain. Har route bataata hai use kaunsi permission chahiye. Built-in roles ke ilawa school apne custom roles bana sakta hai, aur role ko sirf ek campus tak mehdood kar sakta hai.

- `packages/shared/src/permissions.ts` — the permission catalogue (143 permissions) and the six built-in roles (`SYSTEM_ROLES`). Shared, so the web hides what a user can't do.
- `src/rbac/` keeps the database's role catalogue in sync with that file on every migration.
- Routes use `requirePermission('fees.manage')` or `requireAny(...)` from `middleware/auth.ts` .
- **Relationship rules** sit on top of permissions: a teacher may only mark attendance for their own sections; a parent sees only linked children (and only grades/fees/messages their link allows); a campus-scoped role sees only its campuses (`src/lib/structure.ts`).
- **Every rule has a denial test** — a test that tries the forbidden thing (another teacher's class, another institution's student, a student calling a staff route) and expects 403/404. Writing the "should fail" test is as important as the "should work" one.

---

8. The business features, one by one

> **Khulasa:** Is chapter mein har bara feature aur us ki asal "technique" hai — jaise QR attendance kaise chori se bachta hai, fees ka ledger kaise hamesha sahi rehta hai, payment webhook dobara aaye to kya hota hai, waghera.

Structure and people

`routes/structure.ts`, `people.ts`, `imports.ts`, `documents.ts`, `yearend.ts`. Bulk imports parse CSV/Excel (`lib/import-parse.ts`), run a **dry run** that reports errors per row, then apply in one transaction. Custom fields per tenant (`lib/custom-fields.ts`). Year-end promotes students in bulk. ID cards and certificates are PDFs (pdfkit) with QR codes that a public page verifies.

Teaching (LMS)

`routes/courses.ts`, `assessments.ts`, `quizzes.ts`, `gradebook.ts`, `content.ts`, `lib/lms/*`.

- Courses have modules and items (pages, files, video, links, SCORM); **drip release** by date or prerequisite.
- Quizzes support 12 question types (`packages/shared/src/quiz.ts`), pools and randomization, auto-grading, item analysis. **Secure exam mode** (`lib/lms/secure.ts`): time windows, shuffling, one question at a time, focus-loss logging, IP ranges, access codes.
- The gradebook computes weighted categories, drop-lowest, overrides, and letter/GPA grades through grading schemes (`lib/grading.ts`).
- SCORM 1.2/2004 packages run in a sandboxed iframe; xAPI statements are stored.

Attendance and timetable

`routes/attendance.ts`, `timetable.ts`, `bookings.ts`.

- **Rotating QR check-in**: the code shown on the teacher's screen is `HMAC-SHA256(secret, floor(now / 20s))`, cut to 10 characters, and changes every 20 seconds; the server accepts the current and previous window. A photo of the code soon stops working, and the comparison uses `crypto.timingSafeEqual`. An optional **geofence** checks the student's location with the haversine formula.
- Thresholds (e.g. 75 %) feed exam eligibility; holidays block attendance and due dates.
- Automatic timetable generation respects teacher availability, rooms and clashes; room bookings refuse overlaps and timetabled periods.

Exams and results

`routes/exams.ts`, `lib/exams/results.ts`: exam series, seating, admit cards, marks entry, moderation (teacher → head → controller), publishing, merit lists, transcripts — board-style or university-style.

Fees and payments

`routes/finance.ts`, `payments.ts`, `lib/finance/*`.

- A **ledger** (`lib/finance/ledger.ts`): invoices and payments are rows; balances are computed, not stored as a single editable number. Partial payments, refunds, waivers, fines, discounts.
- **Payment providers behind one interface** (`lib/finance/providers.ts`): `createCheckout` , `parseCallback` , settlements. JazzCash (hash over sorted fields with the integrity salt), Easypaisa (inquiry API), Stripe (signed webhooks), bank transfer (reconciled from a statement).
- **Idempotent webhooks**: every provider event is stored by its event id; the same event twice answers `duplicate` and changes nothing.
- **Country decides the gateways** (`packages/shared/src/countries.ts` , `lib/country.ts`): Pakistan → JazzCash and Easypaisa, everywhere else → Stripe; bank transfer everywhere. The currency locks once fee invoices exist.
- Each institution's merchant credentials are **sealed** (AES-GCM) and never shown again.

Admissions

`routes/apply.ts` , `admissions.ts` : a public branded form, documents, application fee, enquiries CRM, entrance tests, merit list, and one click to turn an applicant into a student with guardian accounts.

Reach: notifications, messaging, meetings

`lib/notify.ts` , `lib/messaging.ts` , `lib/conversations.ts` , `routes/messages.ts` , `routes/reach.ts` .

- Notifications go to in-app, email and web push (all free) per person's preferences, with quiet hours, digests and delivery logs with retry. WhatsApp is built but off by default (`WHATSAPP_DRIVER=none`) because it costs money per message; SMS was removed for the same reason.
- **Safeguarding**: an adult and a student under 18 can't have a private conversation unless the institution allows it; guardians are copied; every chat is kept and reportable; a profanity filter (`lib/profanity.ts`).
- Parent–teacher meeting slots, help-desk tickets, emergency broadcasts on every channel.

Optional modules

`routes/modules.ts` : library (copies, loans, fines, barcodes), HR & payroll (salary structures, payroll runs, payslips, leave), transport, hostel, inventory, visitors — switched on per institution (`settings.modules`) and allowed by the plan.

The platform business

`lib/platform.ts` , `routes/platform-saas.ts` , `routes/site.ts` : plans with limits and features, entitlements checks (`assertFeature` , `assertStudentCapacity`), monthly invoices with tax, dunning (reminders, then pause), daily usage metering with cost and margin, tenant health, full ZIP exports (`lib/zip.ts` , a dependency-free ZIP writer), deletion with a grace period, white-label branding and custom domains verified by DNS.

Integrations

- **REST API v1** with scoped API keys (stored as SHA-256 hashes) — `routes/api-v1.ts` .

- **Webhooks** signed like Stripe's: header `t=<timestamp>,v1=<HMAC>` — `lib/webhooks.ts`, with retries and back-off.
- **LTI 1.3** tool launches: RS256-signed `id_token`, a JWKS endpoint, single-use launches — `lib/lti.ts`.
- **OneRoster** CSV export, Google Classroom import, ICS calendar feeds, Zoom/Teams/Meet.

Privacy

`lib/privacy.ts`: consents (guardians for minors, AI features), data requests (copy, correction, deletion), retention periods per tenant, cookie notice, legal pages (privacy, terms, DPA, sub-processors).

---

9. Background jobs, schedules and live updates

> **Khulasa:** Jo kaam request ko dheema kare (email bhejna, reminders, raat ka hisaab) woh "job" ban kar queue mein jata hai. Queue isi PostgreSQL mein hai (pg-boss) — alag Redis nahi. Browser mein live updates PostgreSQL ke LISTEN/NOTIFY se server-sent events ke zariye aate hain.

- **Defining a job** — `defineJob({ name, cron?, run })` in `src/jobs/queue.ts`; `enqueue(name, data, { singletonKey })` queues one. Inside a request the job is written **in the request's transaction**, so it exists only if the request's work commits.
- **Drivers** — `JOBS_DRIVER=pgboss` in real use; `inline` in tests (jobs kept in memory and run with `drainJobs()`, so tests are fast and deterministic).
- **Schedules** — jobs with `cron` (e.g. `recycle.purge` at 02:30, `reminders.due` hourly). pg-boss runs them (`JOBS_SCHEDULE=internal`), or, where the server may sleep (Cloudflare Containers), an outside scheduler calls `POST /api/internal/cron` with a secret every five minutes and the API queues what fell due (`src/lib/cron.ts` is a small cron matcher).
- **Live updates** — `src/lib/events.ts`: code calls `publish()`; the message goes out with PostgreSQL `NOTIFY` *after commit*; every API process `LISTEN`s and forwards it to the open `/api/events` streams (server-sent events) of the right people. Works with several API processes because Postgres is the hub.

---

10. Files and storage

> **Khulasa:** Uploads ke liye ek "storage driver" hai: local disk (development) ya S3-compatible (MinIO, Amazon S3, Cloudflare R2). Code ko farq nahi padta ke file kahan hai.

`src/storage/index.ts` defines `put / get / delete` for a local directory or any S3-compatible store (`STORAGE_DRIVER=s3`, `S3_ENDPOINT` for MinIO or R2). Uploads (`lib/upload.ts`) check size, type and the plan's storage quota, store a SHA-256, optionally scan with ClamAV, and are served only to people allowed to see them (or through short-lived signed links, e.g. scheduled reports).

11. The AI layer

> **Khulasa:** Har AI feature ek hi darwaze (`apps/api/src/ai/client.ts`) se guzarta hai jo on/off switch, budget, model ka intikhab, personal data hatana aur har call ka kharcha likhna sambhalta hai. Model sirf platform owner chunta hai (API management). Tutor jawab nahi deta, samjhata hai, aur sirf course material se baat karta hai.

The gate: `src/ai/client.ts`

Every feature calls `aiText` , `aiJson` , `aiEmbed` or `aiTranscribe` . Each call:

1. checks the off switches — server (`AI_DISABLED`), institution, feature, course, assessment policy;
2. checks the monthly **budget** (plan limit or the institution's lower cap) — `429 AI_QUOTA` ;
3. resolves the provider and model from the platform's configuration (`assignmentFor`);
4. sends the request; **logs tokens, latency and estimated cost** of every call, ok or failed, to `ai_usage` ;
5. if the provider says the chosen model no longer exists (404), retries once with the provider's default model, so a model change can't take a feature down;
6. for JSON features, validates the reply with the prompt's Zod schema and asks once for a repair.

API management: `src/ai/catalog.ts` and `src/routes/platform-ai.ts`

Only the platform owner chooses models. The catalog lists known Gemini and Groq models with their capability (text, embeddings, speech-to-text) and price; "Get latest models" fetches each provider's live list. **Saving a model first makes a tiny test call** — a model that doesn't answer is never saved. Provider keys are sealed in `platform_settings` . The configuration is cached for 10 seconds, so a change applies without a restart. Changing the embeddings model queues `kb.reindex_all` , and until a course is re-embedded its search uses words only — vectors from different models are never compared (`embedding_model` is stored with each chunk).

Providers: `src/ai/providers.ts`

Plain HTTPS to Gemini, Groq (OpenAI-compatible API), and older OpenAI/Anthropic setups — no SDKs. Details that matter for correctness:

- **Gemini 2.5** models "think" before answering and the thinking shares `maxOutputTokens` ; a thinking budget is set by the feature's effort (`effortFor`), with room added, or replies come back cut off.
- Groq's reasoning models get `reasoning_effort` ; Groq also runs Whisper for captions.
- A built-in **fake provider** answers deterministically — and naively on purpose (it follows injected instructions), so tests prove the guards keep students safe, not the model's manners.

Guards: `src/ai/guard.ts`

- `redact()` removes emails, phone numbers, ID numbers and the student's name before text leaves.
- `moderate()` refuses self-harm, violence, sexual content (also written in Urdu or Arabic script) and replies with a safe message; `isInjection()` catches "ignore previous instructions" attempts.

- `leaksAnswer()` checks the tutor's reply against the protected answers of open quizzes.
- `checkSql()` (Ask Acadex) allows one read-only `SELECT` over allow-listed `ask_*` views only.

Prompts: `src/ai/prompts.ts`

Versioned prompts (`[task: tutor v4]`) with a common `SAFETY` block. Course and student content is always wrapped in fenced `<<<DATA … DATA>>>` blocks, and the prompt says data is never instructions — the standard defence against prompt injection. Replies are in English.

The tutor and retrieval (RAG): `src/ai/tutor.ts` , `src/ai/kb.ts`

1. Course material (pages, files, assignment briefs, transcripts) is split into chunks and embedded when published or changed (`indexOffering`).
2. A question is embedded and matched by **hybrid search**: 75 % meaning (cosine similarity of embeddings) + 25 % words (Postgres full-text `ts_rank`).
3. The best chunks go into the prompt as numbered sources; the reply must cite them as `[n]` .
4. The assessment's AI policy decides the mode: `not_allowed` (no help), `limited` (hints only), `allowed` (coach). Under-18 students get a stricter, simpler mode.
5. The reply is checked again (moderation, instruction leaks, answer leaks) before it is shown, even while streaming.

Approve before apply

Copilot output (quiz questions, rubrics, outlines, lesson plans, feedback) is saved as an `ai_actions` **proposal**; nothing changes until the teacher approves it. Grading help is saved as an `ai_assisted` draft for the teacher to accept or edit.

Evals

`src/ai/evals.ts` holds 110 tutor cases (answer leaks, prompt injection, unsafe content, citations, off-topic, English replies to questions asked in Urdu, AI policy). `tests/ai-evals.test.ts` runs them in CI against the fake provider; `npm run ai:eval:real` runs them against a real model.

---

12. The web app (frontend)

> **Khulasa:** React app Vite se banti hai. Har page `apps/web/src/pages/` mein hai, routes `App.tsx` mein. API se baat `lib/api.ts` karta hai (token khud refresh karta hai), data TanStack Query se aata hai, aur menu user ki permissions ke hisaab se banta hai.

- **Entry** — `main.tsx` creates the `QueryClient` (30 s stale time; no retry for 4xx errors), the providers (auth, theme, toasts) and the router. `App.tsx` declares every route; pages are **lazy-loaded** so a student never downloads the platform owner's screens.
- **API client** — `lib/api.ts`: adds the access token, sends `X-Acadex-Client: web`, and on a `401` refreshes the session once (one refresh at a time for all requests) and retries. Errors become `ApiError` with the server's message and field errors.
- **Auth** — `lib/auth.tsx`: restores the session on load with `/auth/refresh`, renews it a minute before expiry, signs out everywhere via `BroadcastChannel`, and clears the query cache **only if someone was signed in** (Chapter 17 explains why).
- **Data** — every screen uses `useQuery` / `useMutation`; `lib/queries.ts` has `usePagedList`. Components always handle loading, empty and error states.
- **Forms** — `react-hook-form` with `zodResolver(sharedSchema)`: the same validation as the API.
- **Layout and menu** — `components/AppShell.tsx` builds the menu from the user's permissions and switched-on modules; `RequireAuth` guards routes.
- **Design system** — `components/ui/` (buttons, fields, dialogs, data tables, tabs, toasts) on Tailwind with semantic colour tokens (`--primary`, `--surface`, …) for light and dark themes.
- **Text** — strings are written as `t('key', 'English text')` (i18next) — English only today, but a language could be added as one JSON file later.
- **Formatting** — `lib/format.ts` formats money, dates and numbers with `Intl` in the institution's currency, timezone and region; the week start follows the country.
- **PWA** — `public/manifest.webmanifest` + `public/sw.js`: installable, app shell and hashed assets cached, some API reads kept for offline, saved content offline, web push, caches cleared on sign-out. **Low-bandwidth mode** (`lib/device.ts`) stops autoplay and shrinks uploads.
- **Phones** — every screen is tested at 360 px width; the floating feedback button moves into the menu on small screens so it never covers buttons.

---

13. The shared package

> **Khulasa:** `packages/shared` woh jagah hai jahan API aur web dono ke "qawaneen" likhe hain — taake form aur server ek hi validation istemal karein aur kabhi alag na hon.

`packages/shared/src` : request schemas (`schemas/*.ts`), types (`types.ts`), the permission catalogue, constants, quiz types, the country list (currency, timezone, week start, gateways), the in-app changelog and help articles. The API bundles it at build time; the web imports it directly. Lesson: define a rule once and import it everywhere.

14. Testing

> **Khulasa:** Teen qisam ke tests: (1) API tests asli PostgreSQL ke saath (har test ke liye taza database), (2) web ke unit tests, (3) Playwright E2E jo asli browser mein desktop aur phone size par poori kahaniyan chalate hain. Har security rule ka "inkaar" wala test bhi hai.

API tests (Vitest + Supertest)

- `tests/helpers/globalSetup.ts` starts an **embedded PostgreSQL**, runs every migration and creates the non-superuser role `acadex_app`, so RLS is really enforced in tests.
- `tests/helpers/fixture.ts` resets the data and builds two institutions (A and B) with users of every role and their tokens — perfect for cross-tenant denial tests.
- Jobs run inline, payment/WhatsApp/AI providers are fakes, email goes to a test outbox.
- Kinds of tests worth copying: denial tests for every permission rule, idempotency tests for webhooks, a test that every route file is mounted and documented, the schema-drift test, AI evals, and config tests.

Web unit tests

Vitest + jsdom for pure logic: the API client, formatting, the English-only check.

End-to-end (Playwright)

`e2e/*.spec.ts` drive a real Chromium on desktop (1366 px) and phone (360 px): the LMS journey, the fee cycle with online payment, the AI tutor and copilot, messaging and meetings, every page of every role, the public site, and **WCAG 2.2 AA accessibility audits with axe-core**.

Habits that paid off

- Write the failing case first (the denial test, the duplicate webhook).
- When an E2E test fails, read the screenshot and the trace before changing anything — several "test problems" were real bugs (Chapter 17).
- Make tests independent of earlier runs (create your own data; don't rely on saved state).

---

15. Security, layer by layer

> **Khulasa:** Security ek cheez nahi, tah dar tah hifazat hai. Neeche har tah aur us ki jagah.

Layer	What	Where
Transport and headers	helmet, CORS allow-list, `no-store` on API answers	`app.ts`
Abuse	rate limits on login, MFA, reset, sign-up, AI	`app.ts`
Input	Zod on every body and query; parameterised SQL; HTML sanitised (DOMPurify on the web, server-side cleaning)	routes, `lib/html.ts`
Identity	bcrypt, short JWTs, rotating refresh tokens with reuse detection, CSRF header, 2FA for admins	`auth/`
Authorization	permissions + relationship rules + denial tests	`middleware/auth.ts`, tests
Tenant isolation	query filters + Row-Level Security	`db/context.ts`, migrations
Secrets at rest	AES-256-GCM sealing for 2FA secrets, payment and AI keys; hashes for API keys, recovery codes, refresh tokens	`auth/crypto.ts`
Signatures	HMAC with `timingSafeEqual` for webhooks, QR codes, cron; RS256 for LTI	`lib/webhooks.ts`, `lib/lti.ts`
Audit	triggers on writes, explicit audit of sensitive reads, platform audit log	migrations, `lib/audit.ts`
Supply chain	`npm audit`, gitleaks in CI, `scripts/leak-scan.mjs` before every commit	`.github/workflows/ci.yml`
Backups	encrypted dumps and a restore drill in CI	`ops/`

16. Build, CI/CD and deployment

> **Khulasa:** API `tsup` se ek bundle banti hai, web Vite se static files. Har push par GitHub Actions lint, typecheck, saare tests, migrations, backup drill aur built server ka smoke test chalata hai. Hosting ke liye Docker images aur Cloudflare dono tayyar hain.

- **Builds** — `apps/api/tsup.config.ts` bundles `server`, `worker`, `db/migrate` and `db/seed` into `dist/` (the shared package is bundled in). `apps/web` builds to `dist/` with Vite.
- **Docker** — `apps/api/Dockerfile` runs `node dist/db/migrate.js && node dist/server.js`; `apps/web/Dockerfile` serves the build with nginx; `docker-compose.yml` adds Postgres, MinIO and Mailpit for local work.
- **CI** (`.github/workflows/ci.yml`) — API: lint, typecheck, all tests, migrations twice on an empty database + seed, backup/restore drill, build, a **smoke test of the built server**, `npm audit`; web: lint, typecheck, tests, build; image builds; the Cloudflare Worker check; gitleaks.
- **Cloudflare** (`cloudflare/`, `docs/cloudflare.md`) — one Worker serves the web app as static assets and forwards `/api/*` to the API running as a Cloudflare Container; files in R2; Neon stays the database; a Worker cron trigger drives the schedules. Deployed by hand (`npx wrangler deploy`) or the manual "Deploy to Cloudflare" workflow.

17. Real bugs found while building — and what they teach

> **Khulasa:** Ye asli ghaltiyan hain jo is project mein pakdi gayin. Har ek se ek sabaq milta hai — inhe parhna kisi bhi tutorial se zyada sikhata hai.

1. **The production server migrated and then closed its own database pool.** `db/migrate.ts` decided "am I the main script?" by comparing `import.meta.url` with the script path. The bundler inlined it into `dist/server.js`, where that comparison was also true — so starting the server ran migrations and called `pool.end()`, and every request failed. Dev mode (unbundled) never showed it. *Lesson:* test the built artefact, not only the source — CI now starts `dist/server.js` and checks `/api/health`.
2. **The pricing page spun for ever for some visitors.** On load, a visitor who wasn't signed in triggered `signedOut()`, which called `queryClient.clear()` — deleting the plans query that was still loading, so the page waited for a result that would never arrive. It only happened when the plans request was slower than the session check. *Lesson:* race conditions hide behind "flaky tests"; reproduce, then instrument (`fetch` logging) until the cause is visible.
3. **API answers were cacheable.** Without `Cache-Control`, browsers kept JSON (including private data) and revalidated it. *Lesson:* set caching headers deliberately; for personal data, `no-store`.
4. **Every Claude request would have failed.** New Claude models reject a `temperature` value and count "thinking" towards `max_tokens`. *Lesson:* read the current API of an external service; your memory of it may be out of date.
5. **`TRUST_PROXY='1'` meant nothing.** Express treats the string `'1'` as an address list, not a hop count. *Lesson:* check how a library parses configuration types.
6. **A guided tour and a floating button covered the page.** The tour overlay blocked clicks; at 360 px the feedback button sat on top of the chat's Send button. *Lesson:* test at phone width with a real browser.
7. **A saved preference undid the user's click.** Low-bandwidth mode toggled on, then the preference arriving from the server switched it off again. *Lesson:* a choice made on the page should win over a stale value loaded in the background.
8. **Changing the currency after invoices exist would corrupt amounts.** *Lesson:* lock data that other records depend on (the currency locks once fee invoices exist).
9. **A committed password from 2024** set off a GitGuardian alert. *Lesson:* scan every commit (`scripts/leak-scan.mjs`, gitleaks in CI) and keep secrets in environment variables or a secret store — never in the repository.

18. Recipe: adding a feature end to end

> **Khulasa:** Naya feature banane ka tay shuda tareeqa. Is ko ek dafa khud kar ke dekhein — masalan "room bookings" (jo isi tareeqe se bana) ko dobara padh kar.

Using **room bookings** as the worked example (`git log --oneline -- apps/api/src/routes/bookings.ts`):

1. **Shared schema** — add the request schema to `packages/shared/src/schemas/…` (here `roomBookingBody`).
2. **Database** — add the table to `apps/api/src/db/schema/…` , run `db:generate` , append RLS + audit SQL to the new migration (see `drizzle/0021_room_bookings.sql`).
3. **Route** — a new file in `apps/api/src/routes/` with `requirePermission` / `requireAny` , `institutionId(req)` , Zod parsing, parameterised SQL; mount it (`routes/lms.ts` or `app.ts`).
4. **Tests** — the happy path, the conflicts, and the denials (wrong role, other institution) (`tests/checklist-gaps.test.ts`).
5. **API docs** — run `npx tsx scripts/gen-openapi-lms.ts` (from `apps/api`) and add the file to the test map.
6. **Web** — a page with `useQuery` / `useMutation` (`pages/office/Extras.tsx` → `BookingsPage`), a route in `App.tsx` , a menu entry in `AppShell.tsx` guarded by a permission.
7. **E2E** — add the page to the page sweep (`e2e/pages.spec.ts`).
8. **Docs** — CHANGELOG, `docs/checklist.md` , and the relevant guide.
9. **Commit** in small steps, running `npm run secrets:check` before each.

---

19. Glossary

Term	Meaning
Multi-tenant	one installation serves many customers (tenants), each isolated
RLS	Row-Level Security: database policies that filter rows per tenant
JWT	JSON Web Token — a signed token carrying who the caller is
Refresh token rotation	each refresh returns a new refresh token; reuse means theft
CSRF	cross-site request forgery; blocked here by a required custom header
TOTP	time-based one-time password (authenticator apps)
AES-256-GCM	authenticated encryption: secret and tamper-evident
HMAC	a keyed hash used to sign messages (webhooks, QR codes)
PKCE	an OAuth/OIDC protection against stolen authorisation codes
Idempotent	doing it twice has the same effect as once
Advisory lock	a named Postgres lock used to serialise work (migrations)
AsyncLocalStorage	Node feature giving each request its own context
SSE	server-sent events: a one-way live stream from server to browser
RAG	retrieval-augmented generation: give the model relevant sources, ask it to cite them
Embedding	a vector of numbers representing meaning, for similarity search
Prompt injection	text that tries to make a model ignore its instructions
PWA	progressive web app: installable, offline-capable website
E2E	end-to-end test: a real browser using the real app
WCAG	web accessibility guidelines (AA is the usual target)

20. Where to read next

To understand…	Open
the whole request path	`apps/api/src/app.ts`, then `src/db/context.ts`, `src/middleware/auth.ts`
sign-in and sessions	`src/routes/auth.ts`, `src/auth/service.ts`, `src/auth/sessions.ts`
a typical feature	`src/routes/bookings.ts` + `tests/checklist-gaps.test.ts` + `apps/web/src/pages/office/Extras.tsx`
money	`src/lib/finance/ledger.ts`, `providers.ts`, `src/routes/payments.ts`
the AI layer	`src/ai/client.ts`, `catalog.ts`, `tutor.ts`, `kb.ts`, `guard.ts`, `prompts.ts`
the web app's plumbing	`apps/web/src/main.tsx`, `lib/api.ts`, `lib/auth.tsx`, `components/AppShell.tsx`
tests	`apps/api/tests/helpers/`, `tests/auth-session.test.ts`, `e2e/lms-journey.spec.ts`
operations and security	`docs/operations.md`, `docs/security.md`, `docs/cloudflare.md`
what is not built yet	`docs/future-gaps.md`